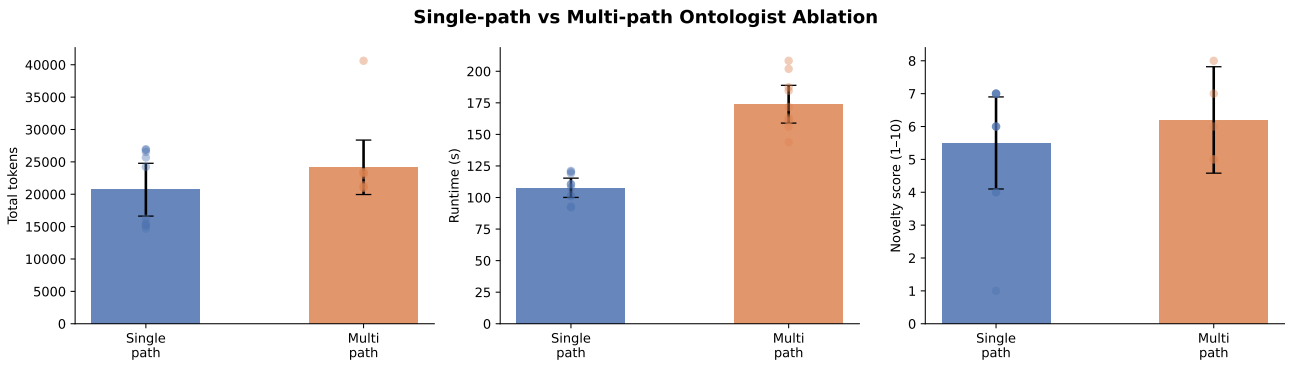

Figure 1. **Single-path vs. Multi-path Ontologist Ablation.** Comparison of single-path and multi-path ontologist settings across three metrics, each evaluated over 10 runs (individual observations shown as dots; bars represent means; error bars denote 95% confidence intervals). *Left*: Total token consumption. The multi-path setting incurs modestly higher token usage on average, with overlapping confidence intervals between the two conditions. *Center*: Runtime. The multi-path setting is substantially slower than single-path, reflecting the cost of exploring multiple ontological paths prior to hypothesis selection. *Right*: Novelty score (1–10). The multi-path setting yields a slightly higher mean novelty score. Overall, multi-path exploration improves output diversity at the cost of increased runtime.

Table 1. **Coding-agent baseline performance on KramaBench with oracle file selection (N=94 tasks).** To establish an upper-bound reference for code generation and execution independently of file retrieval, we evaluated our coding agent on KramaBench—a public general-purpose data-science benchmark of diverse tabular reasoning tasks—under an oracle condition in which all task-relevant files are pre-attached to the model, bypassing the dataset-discovery stage entirely. *Score%* is the composite KramaBench score. *Exact (Reports)* is the fraction of all 94 tasks for which the agent produced the exact correct answer. *Exact (Executed)* is the exact-match rate conditioned on tasks that reached successful execution, isolating answer-quality from execution failures. *Executed%* is the fraction of tasks for which the generated script ran successfully in the sandbox and produced any answer, regardless of correctness; values of 61.7%–78.7% across model backends demonstrate that code generation and execution are performant when provided with correct inputs. Results across four model backends (gpt-5.3-codex, gpt-5.4-03-2026, gpt-5.4-mini, gpt-5.4-nano) confirm that execution success is consistently high under oracle conditions, and that the bottleneck in real-world deployment lies in file retrieval rather than code generation per se (cf. Table 2).

Model	Score %	Exact Count	Exact% (Rep.)	Exact% (Exec.)	Exec. Count	Exec. %
gpt-5.3-codex	42.18	30/94	31.9%	40.5%	74/94	78.7%
gpt-5.4-03-2026	40.79	27/94	28.7%	39.1%	69/94	73.4%
gpt-5.4-mini	32.04	24/94	25.5%	40.0%	60/94	63.8%
gpt-5.4-nano	29.90	19/94	20.2%	32.8%	58/94	61.7%

Table 2. **Coding-agent end-to-end performance on KramaBench with autonomous dataset discovery (N=106 tasks, discovery step budget = 6).** Unlike the oracle baseline in Table 1, the agent here must autonomously search a raw dataset root—with no prior knowledge of which files are relevant—finalize a file selection, and then generate and execute code, all within a fixed step budget. This end-to-end condition tests file-retrieval and code-generation capabilities jointly and represents the realistic deployment setting. Column definitions are identical to Table 1. Execution rates drop to 29.2%–39.6% compared with 61.7%–78.7% under oracle conditions, a reduction of 24–44 percentage points that is attributable to tasks that fail at the discovery stage and never reach code execution. Notably, gpt-5.4-mini and gpt-5.4-nano incur higher token costs than stronger models despite similar or lower execution rates, because these models frequently exhaust the full discovery budget without converging to a valid file selection, wasting inference compute at the retrieval stage. Together with Table 1, these results isolate dataset discovery as the primary performance bottleneck: code generation and execution are competitive when the correct files are provided, but autonomous file retrieval remains the limiting factor for real-world deployment.

Model	Score%	Exact	Exact% (Reports)	Exact% (Executed)	Executed	Executed%
gpt-5.3-codex	18.28	13/106	12.3%	32.5%	40/106	37.7%
gpt-5.4-03-2026	15.76	10/106	9.4%	32.3%	31/106	29.2%
gpt-5.4-mini	12.89	10/106	9.4%	23.8%	42/106	39.6%
gpt-5.4-nano	12.96	10/106	9.4%	25.0%	40/106	37.7%

Table 3. **Three-stage task attrition under autonomous dataset discovery on KramaBench (N=106 tasks per model).** To identify *where* tasks fail in the end-to-end pipeline, we decompose each model's 106 tasks into three sequential stages: (1) *file selection*: discovery successfully finalizes at least one relevant file within the step budget; (2) *execution*: the generated script runs in the sandbox and produces an output; (3) *exact match*: the output matches the gold answer. Tasks that fail at any stage do not proceed to subsequent stages, so attrition is cumulative. The *Dominant failure mode* column identifies the pipeline stage responsible for the greatest absolute task loss per model. gpt-5.3-codex and gpt-5.4-03-2026 lose the majority of tasks at the file-selection stage (60.4% and 68.9% of tasks never produce a valid file set, respectively), confirming that strong reasoning models are bottlenecked by retrieval, not by code generation. In contrast, gpt-5.4-mini selects files for 63.2% of tasks but then fails disproportionately at the execution stage, indicating that aggressive file selection produces noisier inputs that impede successful script execution. Across all models, exact-match rates remain low (9.4%–12.3%), underscoring that improving dataset discovery is the primary lever for advancing overall end-to-end performance.

Model	Files selected	Selected%	Executed	Executed%	Exact	Exact%	Dominant failure
gpt-5.3-codex	42/106	39.6%	40/106	37.7%	13/106	12.3%	No files selected
gpt-5.4-03-2026	33/106	31.1%	31/106	29.2%	10/106	9.4%	No files selected
gpt-5.4-mini	67/106	63.2%	42/106	39.6%	10/106	9.4%	Script exec. failure
gpt-5.4-nano	52/106	49.1%	40/106	37.7%	10/106	9.4%	No files selected

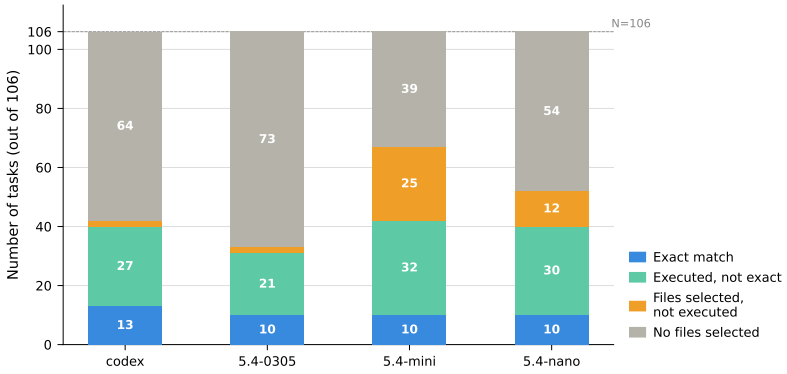

Figure 2. **Task attrition across the three pipeline stages of the autonomous dataset-discovery workflow on KramaBench (N=106 tasks per model).** The coding agent must autonomously identify relevant files from a raw dataset root, generate a solution script, and execute it in a sandboxed environment; tasks that fail at any stage do not proceed further. Each stacked bar shows how 106 tasks are distributed across four mutually exclusive outcomes for a given model backend: **exact match**: script executed and produced the correct answer; **executed, not exact**: script ran but produced an incorrect answer; **files selected, not executed**: discovery succeeded but the script failed or was not attempted; **no files selected**: discovery exhausted its step budget without finalizing any file set. Numbers inside bars are task counts. The dominant gray segment for gpt-5.3-codex and gpt-5.4 shows that these stronger models fail primarily at the discovery stage (60.4% and 68.9% of tasks, respectively), not at code generation. gpt-5.4-mini reaches execution for the largest fraction of tasks (39.6%) by being more aggressive at file selection (63.2% file selection rate), but sustains proportionally higher execution losses, revealing a trade-off between retrieval recall and execution precision. Across all models, the largest single failure category is *no files selected*, establishing dataset discovery as the primary bottleneck for end-to-end performance improvement (cf. Tables 2–3).

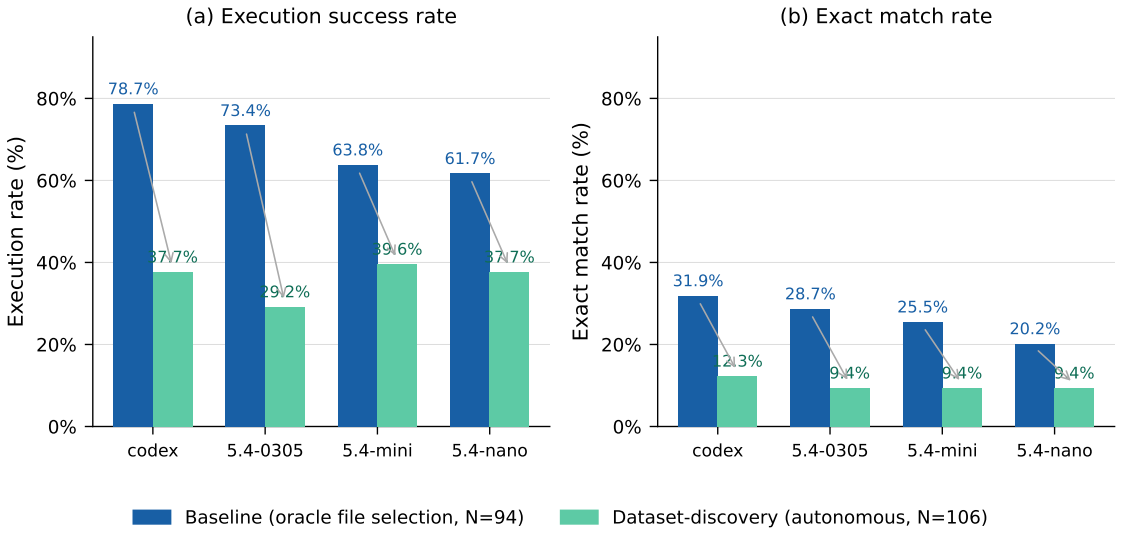

Figure 3. **Impact of autonomous dataset discovery on execution success and exact-match rates across four model backends on KramaBench.** We compare two conditions to isolate the contribution of dataset discovery to end-to-end performance. **Dark blue** bars (*oracle baseline*, N=94 tasks): task-relevant files are pre-attached to the model, eliminating the discovery stage and providing an upper-bound reference for code generation and execution in isolation. **Teal** bars (*autonomous discovery*, N=106 tasks): the agent must search the raw dataset root and finalize a file selection before coding, representing the realistic deployment setting. **(a)** Execution success rate: fraction of tasks for which the generated script ran and produced any answer. **(b)** Exact-match rate: fraction of tasks whose output matched the gold answer. Arrows indicate the per-model drop from baseline to discovery. Execution rates fall by 24–44 percentage points under autonomous discovery, while exact-match rates fall by 10–22 percentage points. Because code generation and execution are competitive under oracle conditions (61.7%–78.7% execution rate), the gap between the two conditions directly quantifies the performance cost of autonomous file retrieval. Improving dataset discovery is therefore the primary lever for closing the gap between controlled and real-world coding-agent performance.

Table 4. **Comprehensive comparison of hypothesis-generation pipelines across automated LLM evaluation and blinded expert oncologist assessment ($n = 10$ runs per pipeline).** *Panel A: Automated novelty evaluation.* Each hypothesis was evaluated by an LLM-based multi-agent ensemble that independently queried three scientific databases (Semantic Scholar, PubMed, arXiv). *Mean novelty score* reports the average rating assigned by all ensemble agents on a 1–10 scale. *Consensus score* measures inter-agent agreement within the ensemble on the same 1–10 scale; higher values indicate that agents converge on the same novelty assessment. *Evaluator confidence* is the ensemble's self-reported certainty (0–100 %) in the aggregated verdict; lower values suggest that the hypothesis is unusual relative to retrievable prior work, which is consistent with higher genuine novelty. *References retrieved*$^{\downarrow}$ counts the distinct literature sources the evaluation ensemble retrieved when searching for prior work similar to each hypothesis. A lower count indicates that the hypothesis has fewer close matches in the indexed literature, serving as a proxy for novelty: hypotheses that are harder to corroborate through retrieval are more likely to explore territory not yet covered by published work. *Panel B: Blinded expert oncologist evaluation.* A board-certified oncologist with *13* years of clinical and translational research experience independently scored each hypothesis on six equally weighted criteria ($w = 1/6$) using a three-point ordinal scale (1 = Poor, 3 = Moderate, 5 = Excellent), with no knowledge of which pipeline generated each hypothesis. The final score is the unweighted mean across all six criteria, $S = \frac{1}{6}\sum_{i=1}^{6} s_i$. All values are mean [95 % CI] across $n = 10$ independent runs (Student's t-distribution, $\mathrm{df} = 9$). Statistical comparisons between SAGE and each baseline were performed using two-sided Mann–Whitney U tests (non-parametric; appropriate for ordinal and small-sample data). * denotes $p < 0.05$ vs. both baselines simultaneously; † denotes $p < 0.05$ vs. ChatGPT only (difference vs. Agent Flow did not reach significance at $n = 10$). **Bold** entries indicate the best-performing pipeline per metric. $^{\downarrow}$Lower values are preferable for this metric. ‡All 10 SAGE hypotheses received a Robustness score of 3 (zero variance; 95 % CI not applicable).

Metric	ChatGPT	Agent Flow	SAGE (ours)
Panel A — Automated LLM novelty evaluation			
Novelty score (1–10)	7.04 [6.57, 7.51]	7.10 [6.73, 7.47]	**7.27 [6.53, 8.01]**
Consensus score (1–10)	7.10 [6.47, 7.73]	7.10 [6.69, 7.51]	**7.30 [6.54, 8.06]**
Evaluator confidence (%)	**84.20 [77.04, 91.36]**	83.00 [76.72, 89.28]	77.60 [70.25, 84.95]
Refs. retrieved$^{\downarrow}$	6.00 [5.25, 6.75]	5.20 [4.20, 6.20]	**2.40 [1.17, 3.63]***
Panel B — Blinded expert oncologist evaluation (scale 1–5)			
Novelty	1.20 [0.75, 1.65]	1.80 [1.06, 2.54]	**4.00 [3.25, 4.75]***
Plausibility	**3.00 [2.33, 3.67]**	2.40 [1.71, 3.09]	**3.20 [2.75, 3.65]**
Feasibility	2.60 [1.70, 3.50]	3.00 [2.33, 3.67]	**3.40 [2.80, 4.00]**
Implementability	1.80 [1.06, 2.54]	2.00 [1.25, 2.75]	**3.00 [2.05, 3.95]**†
Clinical Impact	1.40 [0.80, 2.00]	2.40 [1.43, 3.37]	**3.60 [2.63, 4.57]**†
Robustness	2.80 [2.35, 3.25]	2.60 [2.00, 3.20]	**3.00**‡
Final score (1–5)	2.13 [1.83, 2.43]	2.37 [1.91, 2.82]	**3.37 [2.95, 3.78]***

Revised Figure and Table Captions

Figure 1. SAGE framework architecture. SAGE operates in three stages: a biomedical knowledge graph is constructed from pathology literature; a chain of specialized agents (Path Generation, Ontologist, Scientist, Novelty Critic, Feasibility) traverses the graph to produce biologically grounded hypotheses with an optional human-in-the-loop review; and a Coding Agent translates approved hypotheses into executable analyses on patient cohorts, with results consolidated by a Summary Agent.

Figure 2. Biomedical knowledge graph constructed from 1,650 scientific sources. Nodes (41,053) represent biomedical entities such as genes, diseases, and pathways, color-coded by type; edges (56,338) encode mechanistic relationships extracted from bladder-cancer and computational pathology literature. The zoomed inset highlights the densely connected bladder-cancer subgraph that agents traverse to identify candidate hypothesis paths.

Figure 3. Overall survival stratified by joint FABP5 expression and TLS-like aggregate abundance in TCGA-BLCA. Patients in the high-FABP5 / scarce-TLS group show significantly worse overall survival compared to the low-FABP5 / abundant-TLS group (log-rank $p = 0.0129$; HR = 2.34 [1.25, 4.40]), while the remaining groups fall between these extremes. This hypothesis was autonomously generated by SAGE, rated high novelty and high feasibility, and then empirically validated on the TCGA-BLCA cohort.

Figure 4. Three-phase literature collection and processing workflow used to build the knowledge graph. Phase 1 expands expert seed keywords using OpenAI, MESH, and Semantic Scholar APIs (308 to 1,276 bladder-cancer keywords); Phase 2 crawls high-impact journals in parallel, collecting 137,000 bladder-cancer and 233,000 breast-cancer papers; Phase 3 extracts text from PDFs with a three-tier fallback, converting NCCN graphical guidelines to text via GPT, producing a dataset ready for triple extraction and graph construction.

Figure 5. MCP-based tool orchestration subsystem. The MCP Orchestrator mediates between a coding agent and two domain suites—Medical Oncology (survival analysis, treatment selection) and Histopathology (tumor segmentation, TLS detection, collagen analysis, *etc.*)—by parsing natural-language prompts, selecting appropriate tools, and executing them over shared patient data and whole-slide images. Both Claude and GPT models can interface with the orchestrator.

Figure 6. Tool orchestration subsystem responding to a tumor detection request on a whole-slide image. Upon receiving a natural-language prompt, the MCP Orchestrator invokes the tumor segmentation tool, which processes the WSI and returns a binary mask highlighting detected tumor regions while suppressing non-tumor tissue, along with summary statistics such as the percentage of tumor area detected.

Figure 7. Region-aware patch extraction produced by composing segmentation and spatial reasoning tools. Following tumor segmentation, the subsystem overlays color-coded patches on the WSI—tumor (red), peritumoral (green), and remaining tissue (yellow)—demonstrating how multiple tools are composed into a single coherent workflow to generate structured inputs for downstream analysis.

Table 1. Computational cost and hypothesis quality comparing graph-based (GP) and chat-based (CP) pipeline architectures. In GP each agent receives only context from its immediate predecessor, reducing mean prompt tokens by 57.5% and inference cost by 25–75% relative to CP, where all agents share the full conversation history. Hypothesis novelty drops only marginally (7/10 vs. 8/10), showing that targeted context allocation preserves discovery quality at substantially lower cost.

Table 2. Comparison of hypothesis-generation pipelines via automated LLM evaluation and blinded expert oncologist assessment ($n = 10$ runs per pipeline). *Panel A: Automated evaluation.* An LLM ensemble queried Semantic Scholar, PubMed, and arXiv to score novelty and inter-agent consensus (1–10). Evaluator confidence reflects certainty in the verdict; references retrieved$^{\downarrow}$ counts literature matches—fewer indicates higher novelty. *Panel B: Expert evaluation.* A board-certified oncologist (13 years experience) blindly scored each hypothesis on six equally weighted criteria (1 = Poor, 3 = Moderate, 5 = Excellent); final score $S = \frac{1}{6}\sum s_i$. All values are mean [95 % CI]. $^{*}p < 0.05$ vs. both baselines; $^{\dagger}p < 0.05$ vs. ChatGPT only (Mann–Whitney U). **Bold** = best per metric. $^{\downarrow}$Lower is better. ‡All SAGE Robustness scores = 3 (zero variance).

Table 3. Novelty classification accuracy and score separability across four critic configurations. Moving from a single critic (56.0%) to three independent critics raises accuracy to 81.3%; enabling disagreement-triggered adversarial debate ($\sigma > 1.0$) yields the highest accuracy (91.3%) and the widest score gap between breakthrough and specious proposals (6.78 points), demonstrating that deliberation improves calibration beyond simple score averaging.

Table 4. Knowledge-graph extraction quality evaluated on a stratified sample of 200 triples. Factual grounding (99%) and relation accuracy (100%) remain high before and after refinement. Entity type accuracy is the main limitation, improving from 64.6% to 82.5% strict accuracy after ontology refinement and disambiguation post-processing, with a hallucination rate below 1% throughout.

Table 5. Role, context source, assigned model, and model-selection rationale for each agent in the SAGE pipeline.

Agents with lightweight tasks such as path generation and summarization use cost-efficient models, while agents performing critical scientific reasoning—Scientist, Coding Instructions, and Coding Results Interpreter—use the highest-capability models. This tiered assignment balances inference cost with hypothesis quality at the most consequential steps.

Table 6. Four-step feasibility scoring protocol applied by the Feasibility Agent before empirical validation. Each hypothesis is assessed on data availability (35%), tool maturity (25%), logical soundness and falsifiability (25%), and computational constraints (15%), yielding a weighted score and a categorical verdict (feasible / conditionally feasible / infeasible) to filter out hypotheses that lack concrete data or tooling support before any validation resources are spent.

Table 7. External databases and repositories queried by the Feasibility Agent to ground feasibility judgments. The agent consults seven resources spanning literature (PubMed, Semantic Scholar), omics datasets (GEO, TCGA/GDC), clinical trials (ClinicalTrials.gov), and software ecosystems (PyPI, GitHub, Papers With Code), ensuring that feasibility verdicts are based on verifiable external evidence rather than purely textual reasoning.

Table 8. Full SAGE reasoning and validation workflow for a CXCL13 hypothesis in muscle-invasive bladder cancer. Given a free-form user prompt, SAGE generates a hypothesis linking high CXCL13 expression to improved progression-free interval via TLS formation, assigns a novelty score of 4/10 (moderate; prior TLS–CXCL13 links exist), defines two executable coding tasks, and validates them on TCGA-BLCA, yielding statistically significant survival benefits for CXCL13-high patients (HR ≈ 0.62, $p < 0.01$) with a feasibility score of 8/10.

Table 9. Context provided and withheld for each agent in the specific-memory (graph-based) pipeline. Each agent receives only the output of its immediate predecessor rather than the full conversation history; for example, the Scientist receives only the Ontologist's definitions since path information has already been distilled. This design minimizes token usage, reduces hallucination risk from long prompts, and keeps each agent focused on its specific subtask.

Table 10. Composition and temporal evaluation protocol for the 150-proposal novelty benchmark. Proposals are distributed equally across five tiers from breakthrough discoveries such as AlphaFold2 (Category A, score 8.5–10) to fabricated or retracted claims such as STAP Cells (Category E, score 1–3.5). To prevent temporal leakage, each proposal is evaluated using only literature available before its publication or retraction date.

Table 11. Debate trigger rate, depth, and literature retrieval across the five novelty tiers. Deliberation is triggered most often for breakthrough (76.7%) and specious (83.3%) proposals, where initial critic disagreement is highest. Breakthrough proposals yield few contradictions (0.3 per proposal), consistent with genuine gaps in prior literature, while specious proposals show the highest contradiction rate (8.7), confirming the system's ability to distinguish absence of evidence from evidence of refutation.